LAB 01

Pipeline Skeleton, Configuration and the Alpaca Paper Account

Item	Detail
Topic	Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents
Pipeline stage	Configuration
TSC reference	K5
Duration	20 min
Folder	labs/Lab01-alpaca-setup/
Files	Lab01-alpaca-setup/check_stage.py, pipeline/config.yaml, pipeline/config.py, pipeline/clock.py, pipeline/check.py, pipeline/__main__.py, Lab01-alpaca-setup/solution/, .env.example, requirements.txt
Simulator page	http://localhost:8000/#/settings
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Lion City Capital (fictitious), a Singapore-based systematic trading desk, is building an automated algorithmic trading pipeline for US equities and ETFs on an Alpaca paper-trading account. Every later stage - data, signals, execution, automation - reads one configuration file, so the desk starts by creating that file, keeping the API keys outside the code, and proving that the paper account, the market clock and the data path answer.

Objective

Create the pipeline package and its config.yaml (universe, strategy parameters, risk limits, costs, secret names), build a validating configuration loader and a market-clock module, and run `python -m pipeline check` against the Alpaca paper account or the offline simulator.

Before You Start

- Part A: start the Trading Simulator (uv run trader serve --with-worker, or docker compose up -d) and open http://localhost:8000 > Course Labs > Lab 01. Install steps: Learner Guide, 'Download, Install and Start the Trading Simulator Software'.
- Part B: open a terminal in the labs folder, activate your virtual environment and open your AI coding assistant there; the finished reference code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Part A - In the Trading Simulator

Start the simulator (uv run trader serve --with-worker, or docker compose up -d) and open http://localhost:8000. In the sidebar open Course Labs > Lab 01: the app shows the same steps with a Go button that opens each page pre-filled. Every step works offline on the bundled demo data and the built-in paper broker.

A1. Load the demo data

Open Settings and press 'Load / refresh demo data' (or run `uv run trader seed`). This caches 12 symbols of daily bars 2018-2025, SPY hourly bars, sample headlines, two custom strategies and sample backtests.

Open: http://localhost:8000/#/settings (Settings > Demo data)

Check: Settings shows about 29,000 bars and at least 8 backtests.

A2. Check the integrations

Open Integrations. Without keys, Alpaca shows 'not configured' and the app uses the simulator broker and the offline data. To connect Alpaca paper: alpaca.markets > Paper Trading > API Keys > Generate, copy APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env, restart, then press Test connection.

Open: http://localhost:8000/#/integrations (Integrations)

Check: Alpaca test says 'Paper account ACTIVE' (online) - or you understand the app runs fully offline.

A3. Check the AI agent login

On the Integrations page read the OpenAI Codex card (Sign in with ChatGPT - run `codex login` on your own machine) and the Claude card (API key only - Anthropic's terms do not allow third-party apps to use Claude.ai subscription login). Read both 'Provider terms' panels.

Open: http://localhost:8000/#/integrations (Integrations > AI providers)

Check: You can say which brain the desk will use: codex, claude, openai or the rule-based fallback.

A4. Read the market clock

Look at the US session badge in the top bar. Singapore is 12-13 hours ahead of New York, so a 09:00-18:30 SGT class sits in the US overnight and pre-market sessions.

Open: http://localhost:8000/#/trading (Paper Trading > clock)

Check: You can explain why orders placed during class become extended-hours LIMIT orders.

A5. Tour the Pipeline Map

Open the Pipeline Map and hover each of the 14 stages. Click 'Data ingestion' to see its recent activity.

Open: http://localhost:8000/#/pipeline (Pipeline Map)

Command line: uv run trader status # prints the configuration and which AI providers are available

Check: Data ingestion and Backtesting are green; you can name the stage that enforces the kill switch.

Expected in the app: A working app with demo data loaded, a verified Alpaca paper account or the simulator, and a known AI-agent brain.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

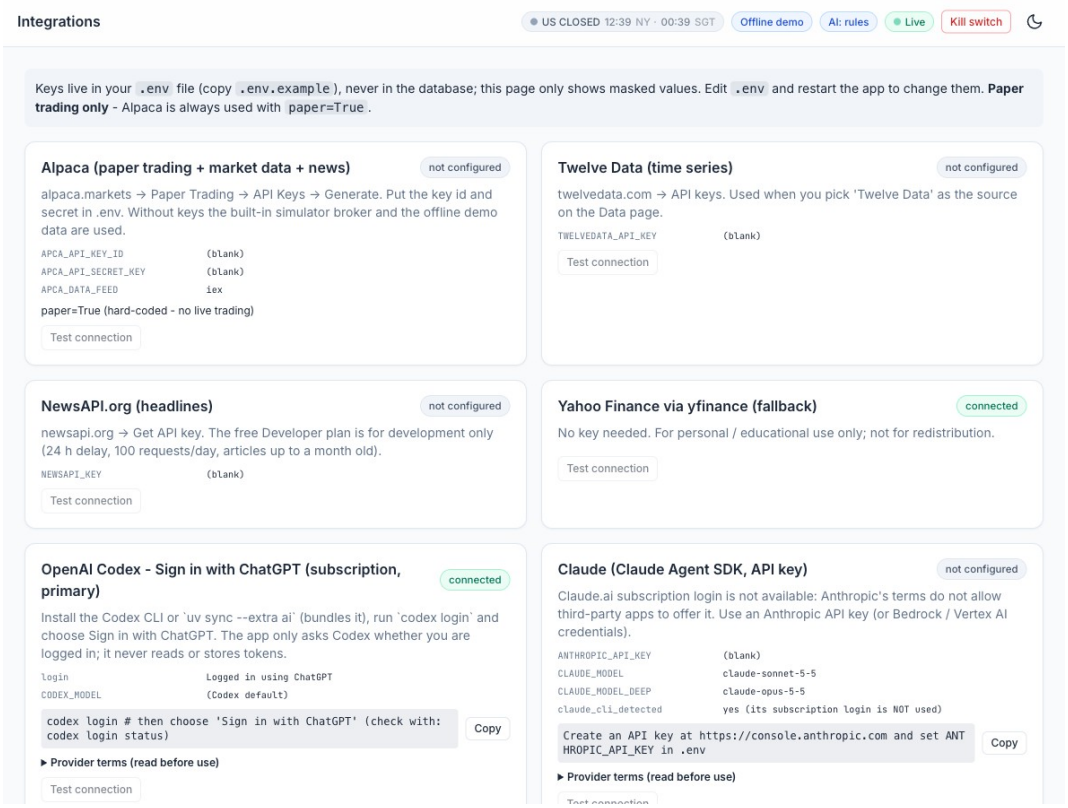


Figure: Lab 01 in the Trading Simulator (Lab01_setup.png).

Part B - In Code with Your AI Assistant

The same pipeline stage in code. AI writes the code; you own the algorithm: paste each prompt into your AI coding assistant in the labs folder (the finished reference code is already in labs/Lab01-alpaca-setup/), run it, and check the result against Part A. The Practical Performance assessment uses this code path.

Step 1: Set up Python and your AI coding assistant (K5)

Open a terminal in the labs folder. Run: python3 -m venv .venv, then source .venv/bin/activate (Windows: .venv\Scripts\activate), then pip install -r requirements.txt. Open the labs folder in VS Code with Claude Code (or your AI coding assistant). Working rule for the whole course: the assistant writes code, YOU review it and run it; never paste an API key into the assistant.

Check: pip finishes without errors and python3 -c "import pandas, yaml, alpaca, sklearn" prints nothing.

Step 2: Store Alpaca PAPER keys outside the code (K5)

Sign up at alpaca.markets, switch to Paper Trading in the account selector, open API Keys and click Generate New Keys. Run: cp .env.example .env and paste the values into APCA_API_KEY_ID and APCA_API_SECRET_KEY. Never put keys in code, e-mail, chat or git. No keys? Leave them blank: every stage then runs OFFLINE on bundled data and the local paper-broker simulator.

Check: labs/.env holds your paper key ID (starts with PK) and secret, or blank values for offline mode.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Write the pipeline configuration (K5)

Ask your assistant for pipeline/config.yaml, then compare it with the reference in Lab01-alpaca-setup/solution/config.yaml. Every later stage reads its symbols, dates, strategy, risk limits and costs from this one file.

AI prompt - paste into your AI coding assistant
In labs/pipeline/ create config.yaml for Lion City Capital (fictitious)'s algorithmic trading pipeline. Sections: mode (auto | offline); broker (name alpaca, paper: true, sim_state_file pipeline_sim.json); secrets_from_env listing only the NAMES of the environment variables APCA_API_KEY_ID, APCA_API_SECRET_KEY, WEBHOOK_SECRET and ANTHROPIC_API_KEY; universe (trade: SPY QQQ AAPL MSFT NVDA JPM XLF GLD TLT; pairs KO/PEP, JPM/BAC, XLF/JPM; benchmark SPY; watch: AAPL as NASDAQ:AAPL in USD and D05.SI as SGX:D05 in SGD); data (start 2018-01-01, end 2025-12-31, timeframes 1Day 1Week 1Month 1Hour, hourly_window 2025-10-01 to 2025-12-31, cache_dir _cache, max_abs_daily_move 0.08); strategy (name ema_cross, symbol SPY, params fast 10 slow 30, execution next_open); costs (preset alpaca_retail, scenarios zero alpaca_retail per_share_broker high_cost, tradingview commission_per_order 1.0 and slippage_bps 0.4); risk (max_position_pct 0.95, max_daily_loss_pct 0.02, max_order_value 100000, max_order_qty 100, allowed_symbols = the trade universe, long_only true, kill_switch_file KILL_SWITCH); live_paper (max_position_pct 0.10, limit_buffer_bps 25, poll_seconds 30, crypto_demo_symbol BTC/USD, crypto_demo_notional 25); webhook (host 127.0.0.1, port 8080, path /webhook, require_signal_agreement false); agents (model claude-sonnet-5-5, symbols NVDA AAPL SPY, max_position_pct 0.20, max_order_value 25000, min_confidence 0.6, stop_atr 2.0, target_atr 3.0); schedule (America/New_York, Asia/Singapore, run_at_exchange_time 09:35). Comment every key. Never put a key or secret VALUE in this file.

Check: config.yaml has the sections broker, secrets_from_env, universe, data, strategy, costs, risk, live_paper, webhook, agents and schedule, and contains no key values.

Step 4: Build the configuration loader with safety validation (K5)

Ask your assistant for pipeline/config.py, then run: python3 pipeline/config.py

AI prompt - paste into your AI coding assistant
Create labs/pipeline/config.py. load_config(path=None, strict=True) reads pipeline/config.yaml with yaml.safe_load (path override from the PIPELINE_CONFIG environment variable), caches it, stores the path in cfg['_path'] and the problems in cfg['_problems'], and raises ValueError when strict and invalid. If mode is offline set LAB_OFFLINE=1. validate(cfg) returns a list of problems: required sections present; broker.paper must be true; risk.max_position_pct and risk.max_daily_loss_pct between 0 and 1; costs.preset must exist in common.backtest.COST_PRESETS; every secrets_from_env value must be an UPPER_SNAKE environment variable name; any string that looks like a credential (PK/AK/CK followed by 14+ capitals or digits, 'sk-ant-', or a 32+ character base64 string) is reported as a secret in the file. Also add apply_cli_flags(argv) (strip --offline -> LAB_OFFLINE=1, and --config PATH -> PIPELINE_CONFIG), secret(cfg, name) that reads the value from os.environ, lab_path(rel), kill_switch_path(cfg), cost_model(cfg, preset=None), tradingview_cost_model(cfg) returning CostModel('TradingView settings', min_commission=1.0, slippage_bps=0.4) from config, and stage_banner(no, title). Use LAB_ROOT from common.config. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python3 pipeline/config.py prints the path of pipeline/config.yaml followed by ': valid'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Add the market clock for US and Singapore time (K5)

Ask your assistant for pipeline/clock.py, then run: python3 pipeline/clock.py

AI prompt - paste into your AI coding assistant
Create labs/pipeline/clock.py. session_at(ts) returns 'regular' (09:30-16:00 New York, Mon-Fri), 'pre' (04:00-09:30), 'after' (16:00-20:00), 'overnight' (20:00-04:00 from Sunday night to Friday 04:00, Alpaca 24/5 trading) or 'closed'; naive timestamps are New York time. RULES maps each session to the allowed order types, time_in_force values and extended_hours flag: regular = market, limit, stop, bracket; pre, after and overnight = LIMIT orders only with time_in_force DAY or GTC and extended_hours=True; closed = a GTC limit order that queues for the next session. broker_now(broker) uses the simulator's replay clock (broker.state['now']) when it has one, otherwise the real UTC clock. market_status(broker=None, now=None, override=None) returns now in America/New_York and Asia/Singapore, the session, a holiday flag and the rules; when the broker is the Alpaca paper client (mode 'alpaca-paper') and the session is 'regular' but broker.get_clock().is_open is False, report 'closed' (exchange holiday). describe(status) returns one line such as 'Mon 2026-10-19 21:05 New York = Tue 09:05 Singapore | session: OVERNIGHT (...)'. Use zoneinfo only. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: One clock line with New York and Singapore time. At a weekend it shows CLOSED; during a weekday 09:00-18:30 Singapore class it shows OVERNIGHT (until 16:00 SGT in US summer time) or PRE.

Step 6: Build the check command and the pipeline CLI (K5)

Ask your assistant for pipeline/check.py, pipeline/__init__.py and pipeline/__main__.py. Compare with Lab01-alpaca-setup/solution/.

AI prompt - paste into your AI coding assistant
Create labs/pipeline/check.py with run(reset=False, quiet=False) returning a dict and main(argv) for `python -m pipeline check [--reset]`. Print: [1] the config path, firm, mode, universe, strategy, risk limits and the validation result; [2] versions of pandas, numpy, scipy, sklearn, xgboost, matplotlib, yaml and alpaca; [3] each entry of secrets_from_env as 'found (PK12...)' for the Alpaca key ID, 'found (hidden)' for any secret, or 'not set' - never print a secret value; [4] the paper account from common.broker.get_broker(state_file=cfg['broker']['sim_state_file'], reset=reset): account number, status, equity, cash, buying power and number of positions; [5] the market clock line from pipeline.clock; [6] the last 3 daily bars of the strategy symbol from common.data.get_bars with data_source(). End with 'RESULT: PIPELINE READY - paper trading only' and exit code 0, or 'NOT READY - <problems>' and exit code 1. Then create pipeline/__init__.py (docstring, __version__) and pipeline/__main__.py with a COMMANDS table {command: (lab, module, function, description)} for all 20 stages (check, validate, ingest, hypothesis, indicators, reconcile, signals, execute, evaluate, search, ml, regime, abtest, pairs, backtest, run, schedule, kill, webhook, alert-sim, diagnose, report, agents, e2e); import the module lazily; strip --offline and --config with apply_cli_flags; print the table when called with no arguments. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline (no arguments) lists 24 commands across Labs 1-20.

Step 7: Run the pre-flight check (K5)

Run: python -m pipeline check --reset (same as: python3 Lab01-alpaca-setup/check_stage.py --reset). Then run echo $? (Windows: echo %errorlevel%) to see the exit code that a scheduler would see.

Check: Offline: validation OK; 8 libraries OK; all four secrets 'not set'; account SIM-LCC-0001, status ACTIVE, equity, cash and buying power USD 100,000.00, 0 positions; a clock line; SPY bars ending 2025-12-31 with close 676.64; 'RESULT: PIPELINE READY - paper trading only'; exit code 0. Online: your PAPER account number and balance and the session from the real Alpaca clock.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Prove the safety rules work (K5)

In pipeline/config.yaml change broker.paper to false and run python -m pipeline check. Restore true. Then paste the fake key PKABCDEFGHIJKLMNOP as the value of firm and run the check again. Restore the firm name and run the check a third time.

AI prompt - paste into your AI coding assistant
Explain to me, as a finance professional new to coding, why labs/pipeline/config.py refuses to run when broker.paper is false or when a value in config.yaml looks like an API key, and what could go wrong at a trading desk without these two checks. Do not change any file.

Check: First run: 'NOT READY - broker.paper must be true ...', exit code 1. Second run: 'firm looks like a secret - move it to an environment variable', exit code 1. Third run: PIPELINE READY again.

Expected Output

python -m pipeline check --reset (offline): configuration valid; 8 libraries OK; 4 secrets 'not set'; account SIM-LCC-0001 ACTIVE with equity, cash and buying power USD 100,000.00 and 0 positions; a clock line in New York and Singapore time (CLOSED at weekends, OVERNIGHT or PRE during a weekday Singapore class); last SPY bar 2025-12-31, close 676.64; 'RESULT: PIPELINE READY - paper trading only', exit code 0. python -m pipeline lists 24 commands.

Guide: Download the Course Materials: Learner Guide, Slides and Lab Manual

Get your own copy of the courseware before Day 1. Everything is free to download.

1. Go to the course portal

Open lms-tms.tertiaryinfotech.com in your browser (the Tertiary Infotech Training Management System).

2. Request a one-time password

Under Sign in to the TMS, enter the email you registered for the course with and click Send OTP. (Prefer a password? Click Login with password instead.)

3. Key in the OTP

Open the email from the TMS, copy the one-time password and key it in. No OTP, or the class is missing after sign-in? Use Register with your Course Run ID (from your trainer or course administrator).

4. Download the Learner Guide and slides

Open this course. Under Courseware, open the Learner Slides and the Learner Guide and download both PDFs (they open in Google Drive: File > Download).

5. Download the lab manual

Open Activities / Lab. It holds the labs folder: every lab has its own folder with a LabNN_Instructions.pdf (the lab manual for that lab) and its Python or Pine Script files. Download the whole folder (right-click > Download).

6. Or download everything from GitHub

Open https://github.com/tertiarycourses/TGS-2023018794-Machine-Learning-101-for-Financial-Trading, click Code > Download ZIP and unzip it. The courseware/ folder holds the Learner Guide (PDF, Word and Markdown), the slides and the Lesson Plan; the labs/ folder holds all twenty lab manuals and files. With Git installed you can instead run: git clone https://github.com/tertiarycourses/TGS-2023018794-Machine-Learning-101-for-Financial-Trading.git

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

7. Keep the materials together

Unzip into one folder on your laptop, for example Documents/ML101-Trading, so the Learner Guide, the lab manuals and the simulator sit side by side.

Each lab folder has its own LabNN_Instructions.pdf - the step-by-step lab manual.

Guide: Download, Install and Start the Trading Simulator Software

The labs run in the course's trading simulator - a user-friendly web app on your own laptop. Install it once before Lab 1. Choose Option A (uv) or Option B (Docker).

1. Install the prerequisites

Install Git and Python 3.11 or later. Then install uv: on macOS or Linux run curl -LsSf https://astral.sh/uv/install.sh | sh ; on Windows (PowerShell) run powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex". For Option B install Docker Desktop instead.

2. Download the software from GitHub

Run: git clone https://github.com/alfredang/algorithmic-trader-bot.git - or open https://github.com/alfredang/algorithmic-trader-bot, click Code > Download ZIP and unzip it. Then open a terminal in the algorithmic-trader-bot folder.

3. Create your settings file

Copy .env.example to .env (macOS/Linux: cp .env.example .env ; Windows: copy .env.example .env). Leave it as it is to use the offline demo data, or add your Alpaca paper keys (and optional Twelve Data / NewsAPI keys) as described in the next guides.

4. Option A - install and start with uv

Run: uv sync (installs everything), then uv run trader seed (loads the demo data, about 15 seconds), then uv run trader serve --with-worker (web app, API and the automation scheduler in one process). Leave the terminal open.

5. Option B - start with Docker Compose

Run: docker compose up -d --build and wait until all four containers are healthy (docker compose ps). Load the demo data with docker compose exec backend trader seed . Stop it later with docker compose down (your data is kept).

6. Open the app

Open http://localhost:8000 in your browser (if port 8000 is busy: uv run trader serve --with-worker --port 8010). The onboarding wizard lets you choose the offline demo or connect your Alpaca paper account; Settings > Load demo data reloads the demo at any time.

7. Connect the AI agents (optional)

For the multi-agent desk: run uv sync --extra ai , then codex login and choose Sign in with ChatGPT (uses your own ChatGPT subscription), or put an ANTHROPIC_API_KEY or OPENAI_API_KEY in .env. Check Integrations in the app. Without either, the desk runs its rule-based agents - every lab still works.

8. Start Lab 1

In the app's sidebar open Course Labs > Lab 1 and follow the checklist alongside Lab01_Instructions.pdf. If the README of the software gives a different command or port, follow the README - it is the authoritative start-up guide.

Simulator: github.com/alfredang/algorithmic-trader-bot - the README has the full start-up steps and troubleshooting.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Guide: Sign Up for Alpaca and Open the Paper-Trading Account

Alpaca paper trading is free and needs no funding. You use it in every lab that places orders. Do this before Lab 1.

1. Create the account

Go to alpaca.markets and choose Sign Up. Register with your email address and a strong password, then open the verification email and confirm your address.

2. Log in to the dashboard

Log in at app.alpaca.markets and go to the Home page.

3. Switch to the paper account

In the upper-left corner choose your 'Paper Trading' account. Paper trading needs no deposit and no brokerage approval; every order uses virtual money.

4. Generate your paper API keys

On the paper account's Home page find the API section and generate a key. Copy the Key ID and the Secret Key immediately - the secret is shown only once. Generating a new key invalidates the old one.

5. Store the keys safely

In the labs folder copy .env.example to .env and paste the two values against APCA_API_KEY_ID and APCA_API_SECRET_KEY. Never put keys in a script, a slide, a chat or any file you share; .env is git-ignored.

6. Check the connection

In the labs folder run: python -m pipeline check. The banner must say ONLINE and show your paper account's status and equity. Paper keys only work on the paper endpoint (paper-api.alpaca.markets) - they cannot touch a live account.

7. If you cannot open an account

Leave .env blank. Every lab runs OFFLINE on the bundled data and the built-in paper-broker simulator, so nothing in the course is blocked.

No funding needed. Paper keys only work on paper-api.alpaca.markets. No account? Every lab also runs offline.

Debrief

A trading pipeline is only as safe as its configuration and its connection to the broker. One validated config file, paper-only enforcement, keys held in the environment and a pre-flight check that a scheduler can read from the exit code show how AI and algorithmic trading programs are connected to execution platforms in a controlled way (K5).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
ModuleNotFoundError: pipeline, common or yaml	Activate the venv, run pip install -r requirements.txt and run every command from inside labs.
401 Unauthorized or forbidden online	The keys are wrong or are LIVE keys. Regenerate PAPER keys and check .env has no spaces or quotes around the values.
Mode shows OFFLINE although keys are set	Remove --offline, set mode: auto in config.yaml and make sure LAB_OFFLINE is not 1 in .env.
The assistant wrote code that prints the secret	Reject the change: only the first 4 characters of the key ID may ever be shown, never the secret.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.